2026-03-10

Center of Innovation for Sustainable Quantum AI

量子計算とテンソルネットワーク

#3: MPSの時間発展

https://github.com/todo-group/tnqc-springschool

量子計算とテンソルネットワーク

- #3 MPSの時間発展
  - MPS（Matrix Product State）
  - 量子系のユニタリ時間発展
  - TEBDアルゴリズムに基づくシミュレーション
  - MPS状態からのサンプリング
  - Qailo: Simple Quantum Circuit Simulator

MPS (Matrix Product State)

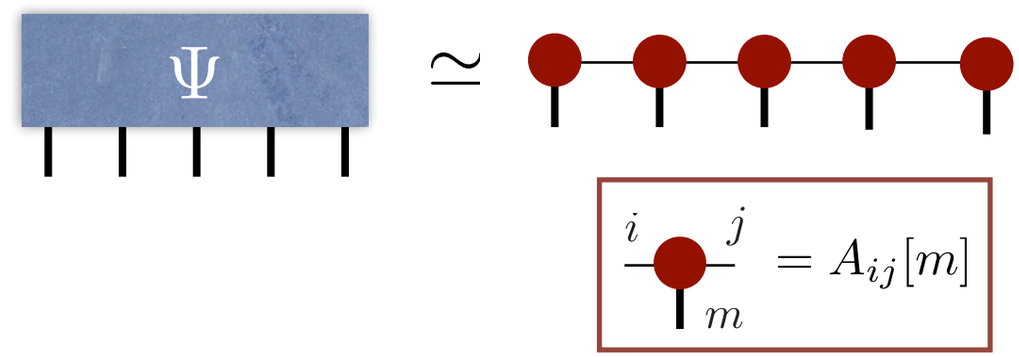
MPS（Matrix Product State - 行列積状態）

- 一次元的につながったテンソルネットワーク
  - 応用数理分野では「Tensor Train」と呼ばれる

- N 本足のテンソル（ベクトル）を行列の積で表現

$$\Psi_{i_1 i_2 \ldots i_N} \simeq A_1[i_1] A_2[i_2] \cdots A_N[i_N]$$

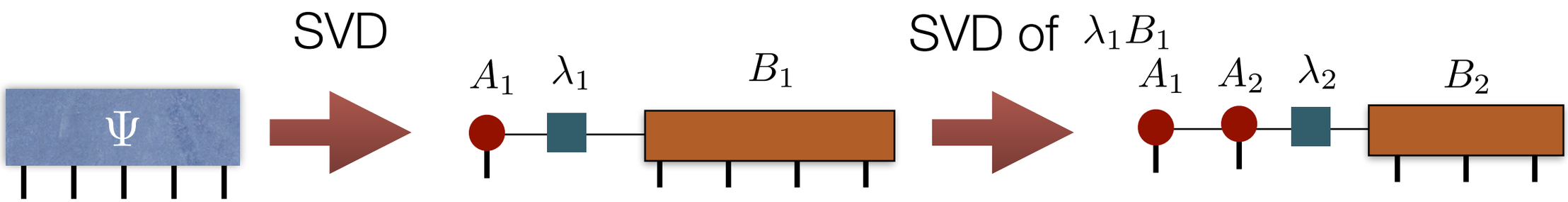
MPS

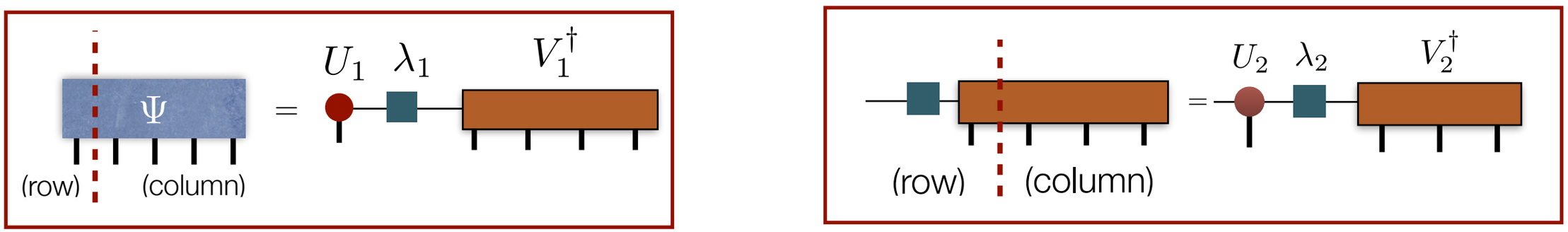


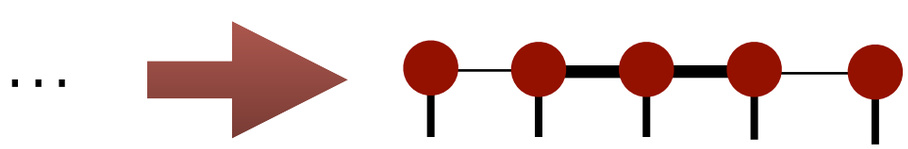
- MPSから外に出ている足を「物理的な足」と呼ぶ
- テンソルの間を繋ぐ足を「仮想的な足」と呼ぶ
- 仮想的な足について縮約をとると、元のテンソルと同じ形に戻る

行列積状態への変換

- 任意のテンソル(ベクトル)は特異値分解を繰り返すことで厳密なMPS表現に常に変換できる



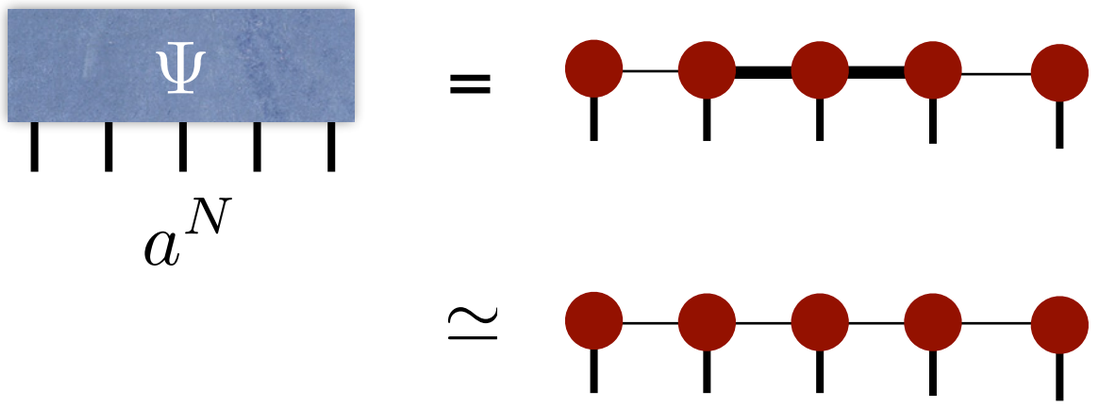




- ボンド次元は場所に依存 → 最大のボンド次元 $= O(a^{N/2})$

低ランク近似

Hands-on: 31_statevector2mps



最大ボンド次元 = $a^{N/2}$

a^N

最大ボンド次元 = χ

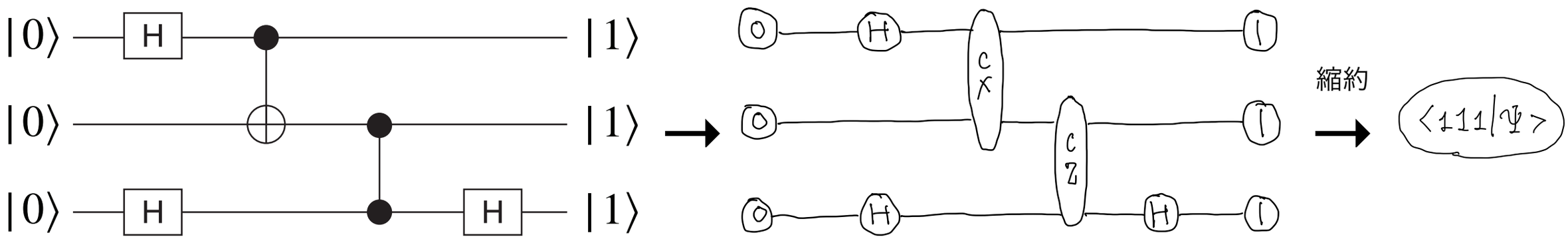
$\propto Na\chi^2$

- ゼロの特異値が現れた場合には、それに対応する行や列は捨ててよい
  - テンソルネットワークによるテンソルの圧縮表現
- 十分に小さな特異値を捨てることで、元のテンソルを精度良く近似可能
  - テンソルネットワークによるテンソルの低ランク近似
  - 指数関数的なデータ量を，多項式程度まで大幅に圧縮できる
  - （どこまで効率的に近似可能かどうかは、表そうとしている「状態」の性質（＝エンタングルメントの強さ）による）

量子系のユニタリ時間発展

量子回路のテンソルネットワークシミュレーション

- 量子回路 ⇒ テンソルネットワーク



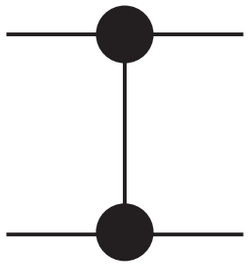
  - 1量子ビットゲート → 2本足テンソル（2x2）
  - 2量子ビットゲート → 4本足テンソル（2x2x2x2）
  - 初期状態・出力状態 → 1本足テンソル(ベクトル)の組
  - 出力状態の振幅 → テンソルネットワークを縮約することで得られる
- 量子回路では必ず左から右に時間が進む
  - テンソルネットワークの縮約はどのような順番で計算しても結果は同じ
- テンソルネットワークの古典コンピュータでの計算は非常にコストが高い
  - 縮約順序を工夫することで計算量を劇的に減らせる可能性
  - 近似的な縮約も可能(低ランク近似、テンソル繰り込み群)

MPO への分解

- ゲージ自由度のため, 分解は一意ではない

$$AB = A(MM^{-1})B = (AM)(M^{-1}B) = A'B'$$

- 行列のランクは, テンソルをどのような行列形に表すかに依存する
  - CZゲートの 2 つの異なる行列表現

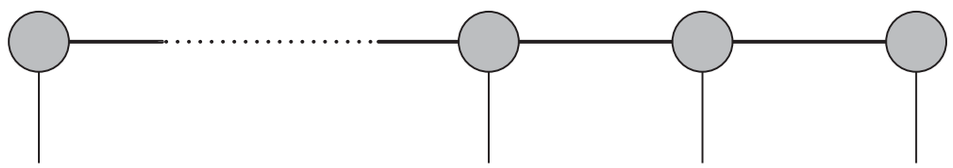


$$U^{\mathrm{CZ}} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & -1 \end{bmatrix}$$

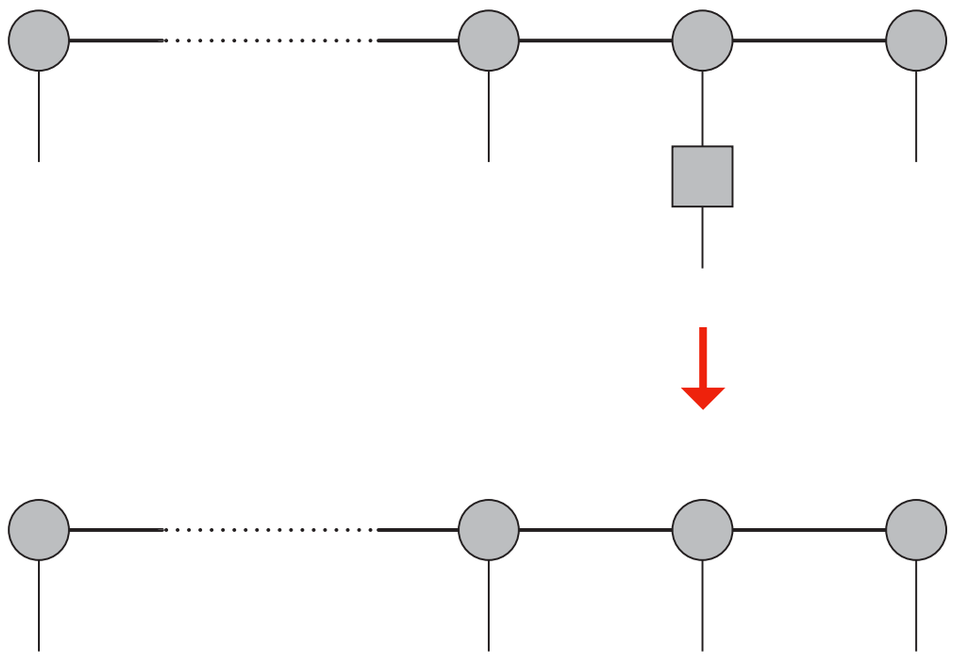
$$M^{\mathrm{CZ}} = \begin{bmatrix} 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & -1 \end{bmatrix}$$

MPSによる量子回路シミュレーション: TEBD

- 波動関数のMPS表現
  - n 量子ビット波動関数をMPS（3本足テンソル- x n）で近似



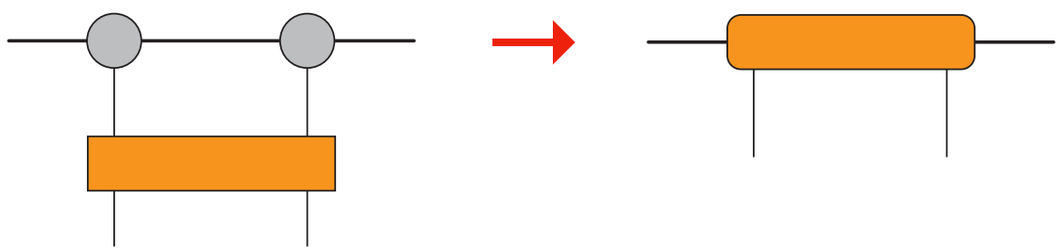
  - 1量子ビットゲート（近似なしで実行可能）



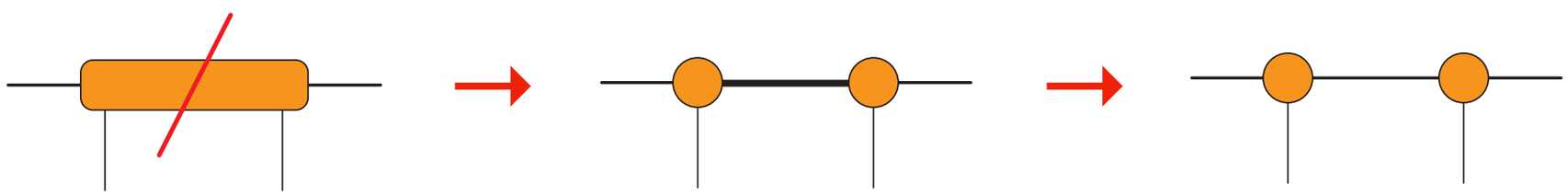
2量子ビットゲート

Hands-on: 32_tebd

- MPSに2量子ビットゲートを適用する（縮約）
  - 2量子ビットゲートを適用すると形が変わる



- テンソルを分解してもとのMPSの形にもどす
  - 分解するだけではボンド次元が増加してしまうので、間をつなぐボンド次元が χ を超えないように低ランク近似する



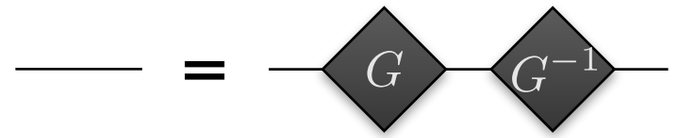
  - 低ランク近似の際にMPS全体が最適化されるように近似すると、精度が飛躍的に向上する→カノニカル形式の利用

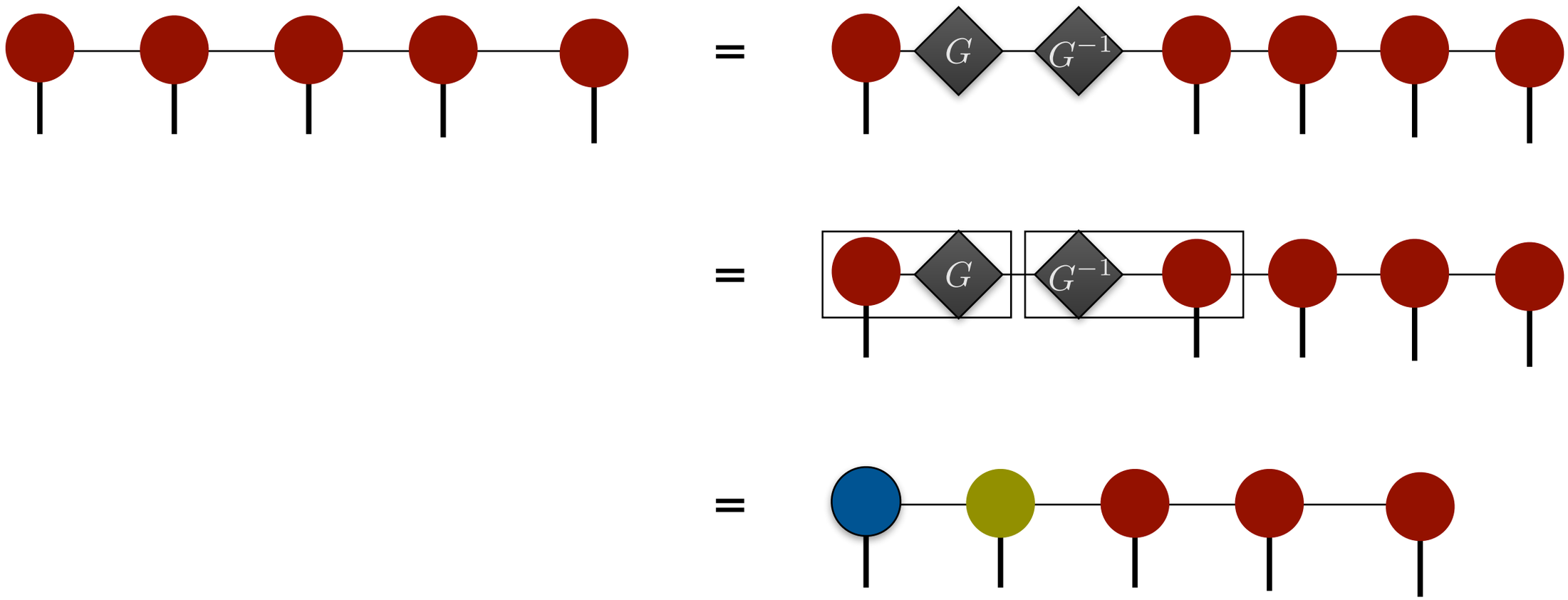
TEBDアルゴリズムに基づくシミュレーション

MPS のゲージ冗長性

- MPS は一意ではない
  - ゲージ自由度がある
  - GG^{-1} という行列を挿入しても, MPS の縮約結果は変わらない

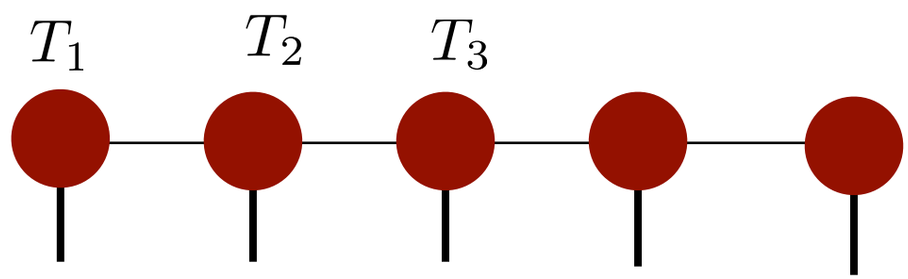
$$I = GG^{-1}$$

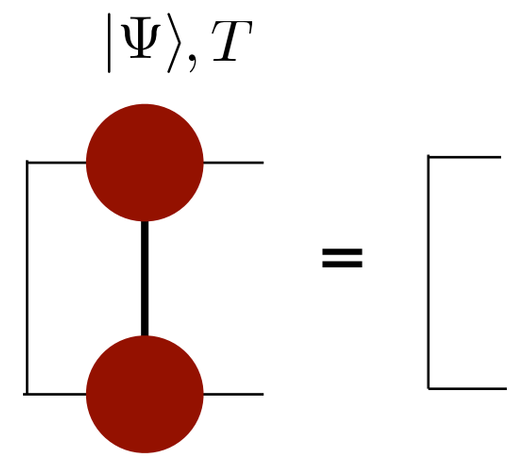




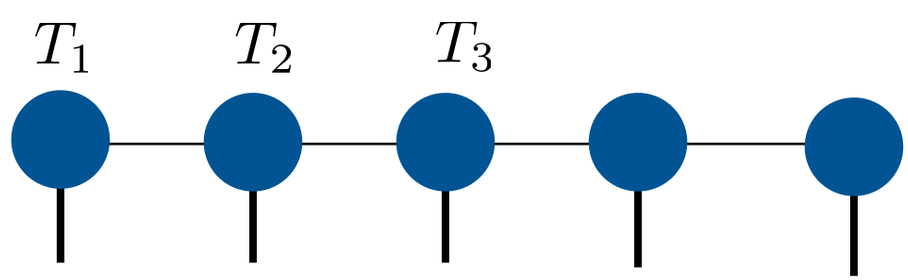
MPSのカノニカル形式(正準形)

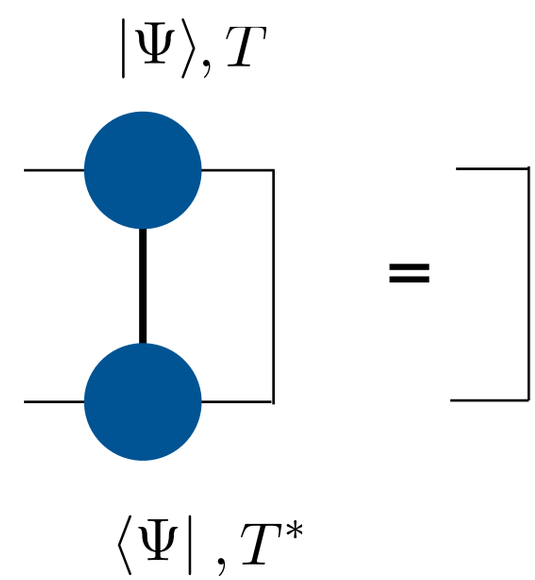
- ゲージ自由度 G を使って MPS を「正準化」できる
  - 左カノニカル形式





  - 右カノニカル形式



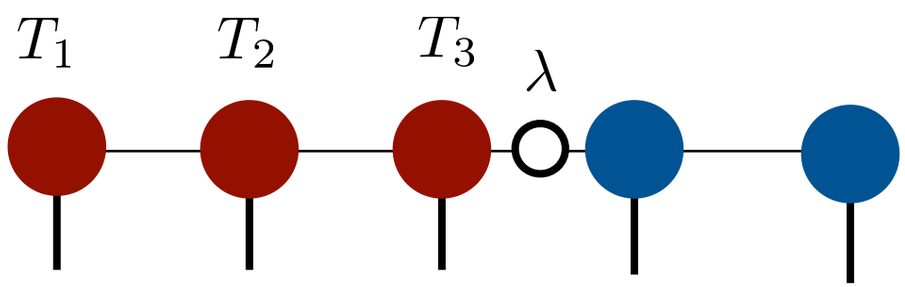


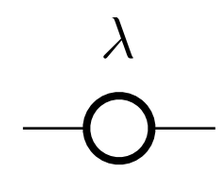
  - GはSVD(または対角化)を使って求められる

Review: U. Schollwöck, Annals of Physics 326, 96 (2011)

混合カノニカル形式

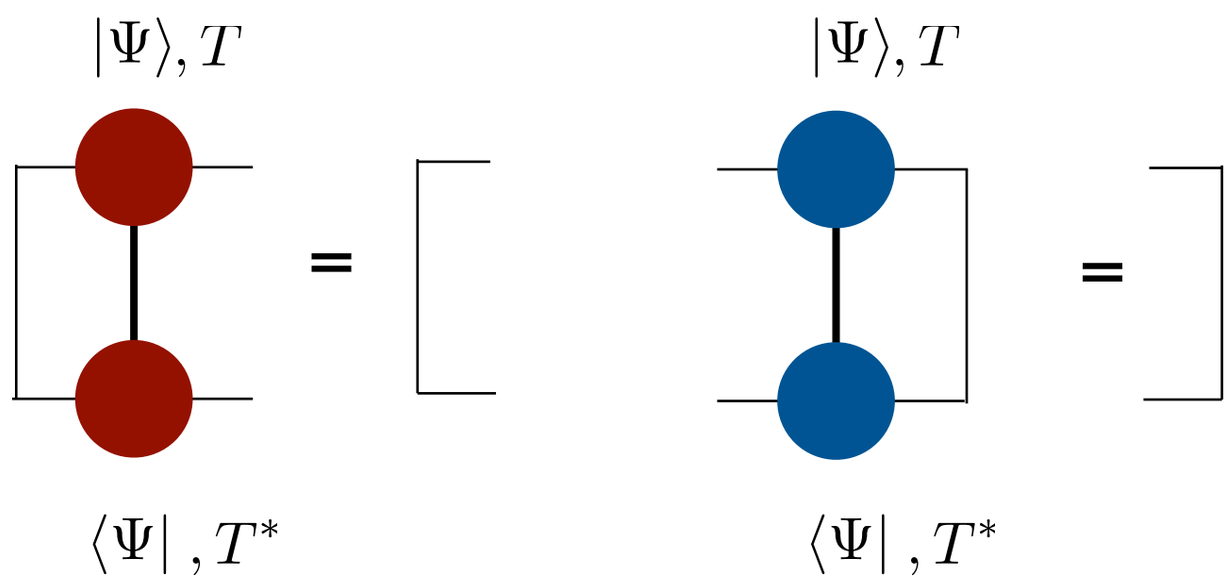
- 混合カノニカル形式





境界にある対角行列は
Schmidt係数に対応する

- 左･右カノニカル条件

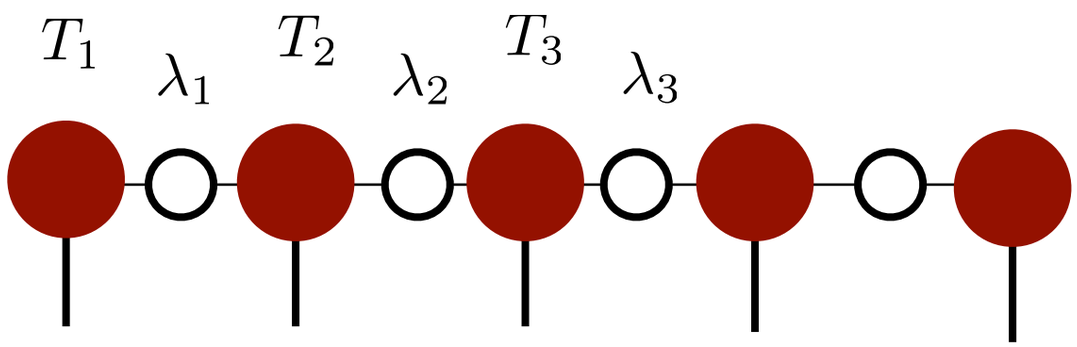


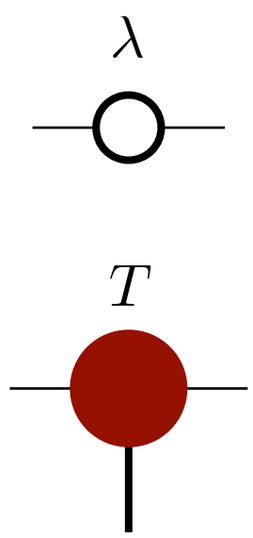
- 左側と右側の境界は SVD を使って動かせる

Review: U. Schollwöck, Annals. of Physics 326, 96 (2011)

Vidalカノニカル形式

- MPSの別のカノニカル形式(Vidalカノニカル形式)

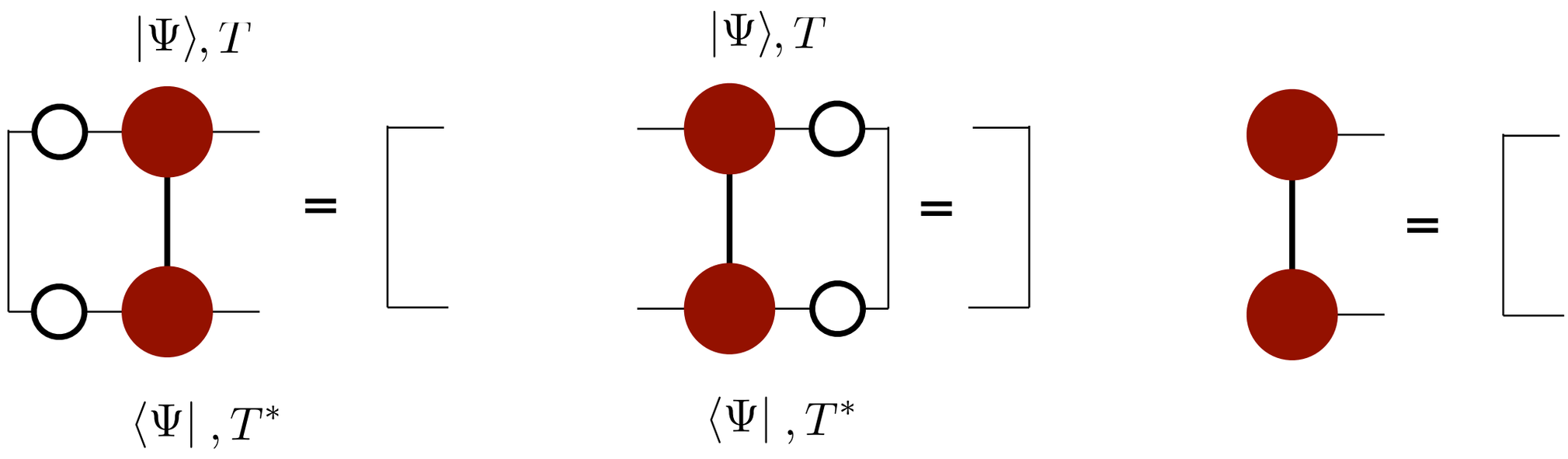




Schmidt係数に対応する対角行列

仮想インデックスは
Schmidt 基底に対応

- 右・左・境界カノニカル条件

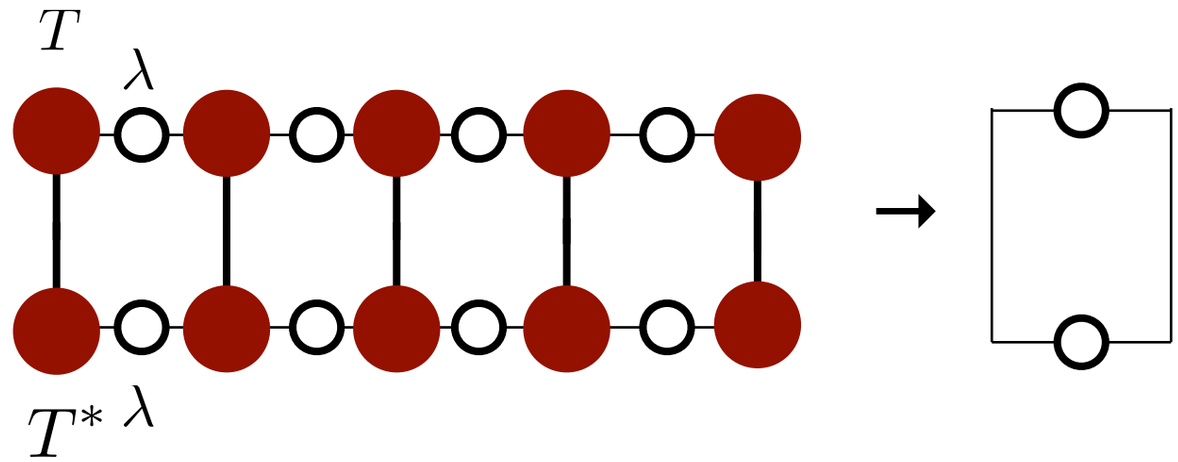


G. Vidal, Phys. Rev. Lett. 91, 147902 (2003)

カノニカル形式のMPSにおける期待値

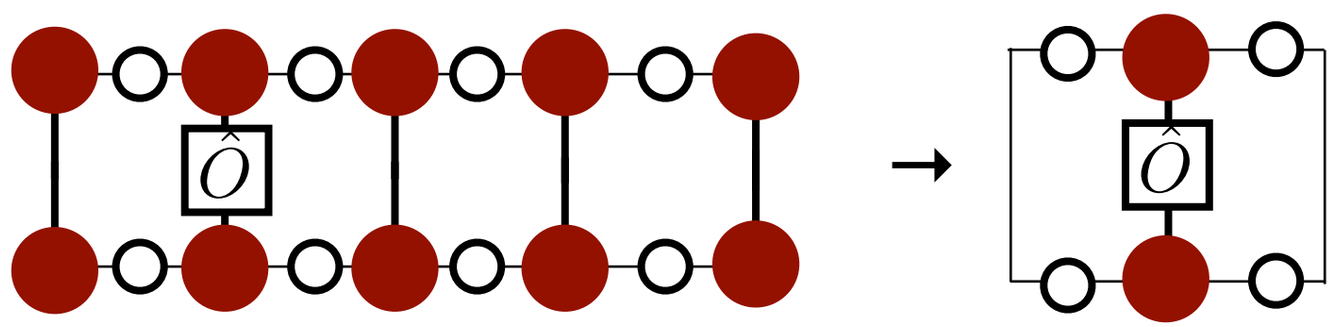
・カノニカル条件のもとでは期待値の評価が非常に簡単になる

$\langle \Psi | \Psi \rangle =$



$= \sum_i \lambda_i^2 = 1$

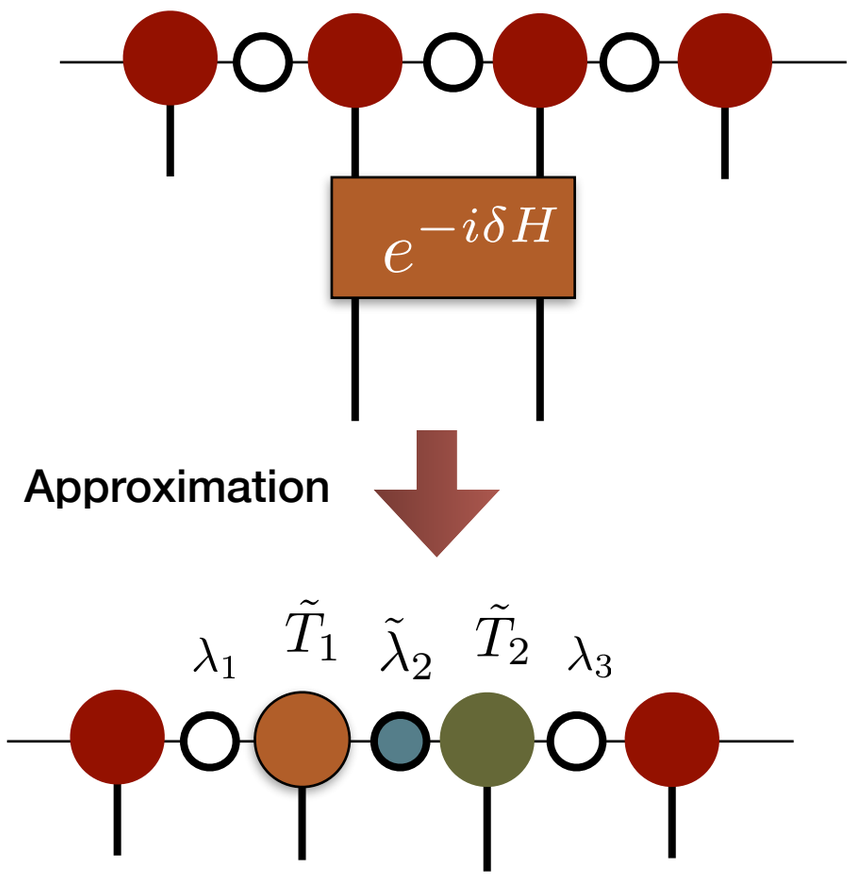
$\langle \Psi | \hat{O} | \Psi \rangle =$



・混合カノニカル形式でも同様に簡単な図式が得られる

TEBD アルゴリズム

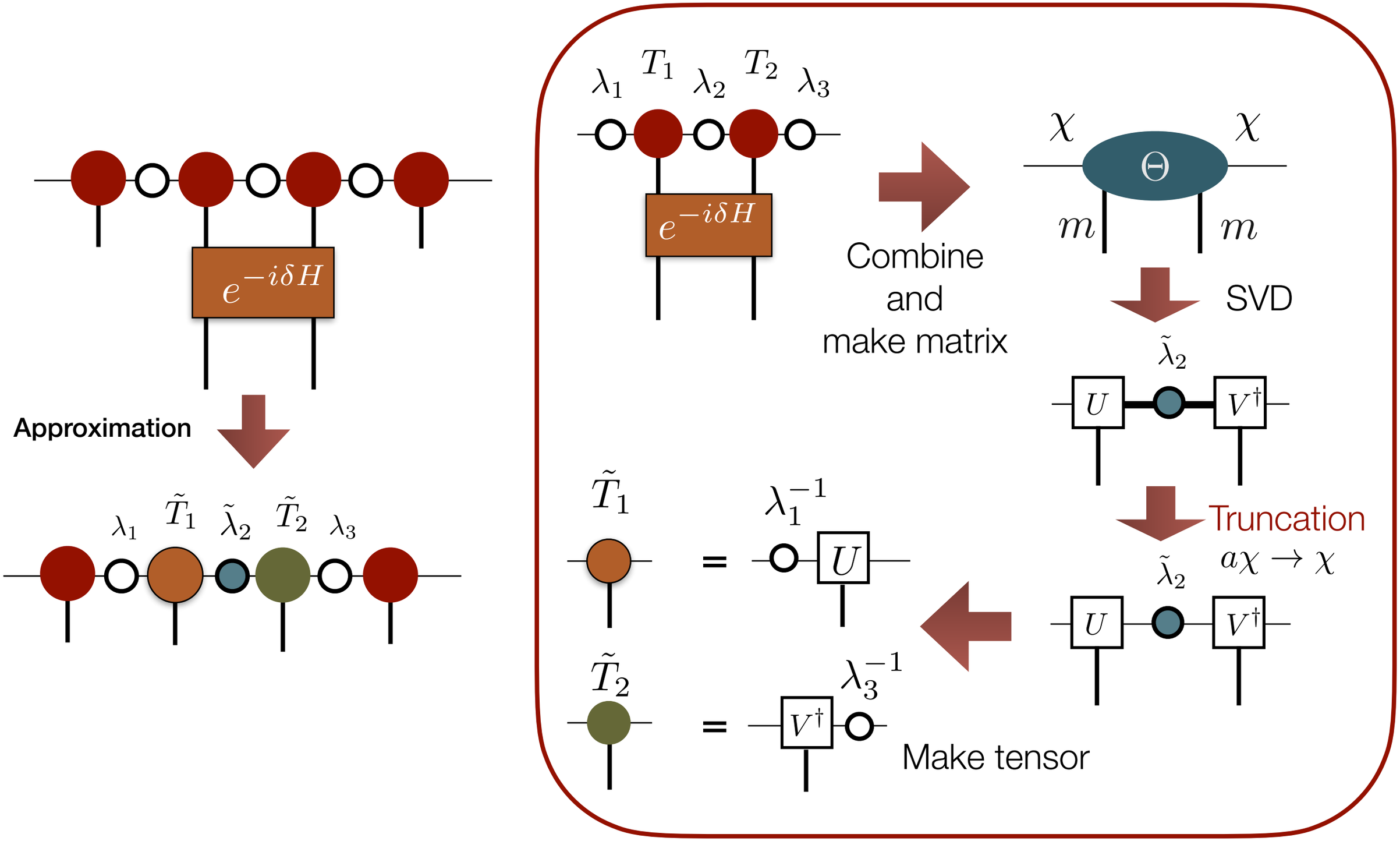
- Time Evolving Block Decimation (TEBD)
  - カノニカル形式のMPSを使って, 局所的に高精度な変換を行う



- 時間発展演算子が直接かかる2つのテンソル(＋その間の対角行列)だけが変化する

G. Vidal, Phys. Rev. Lett. 91, 147902 (2003)

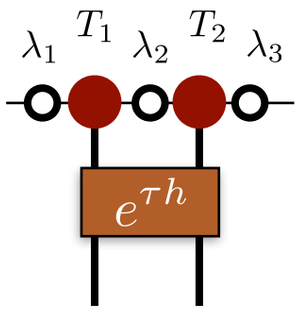
TEBD algorithm



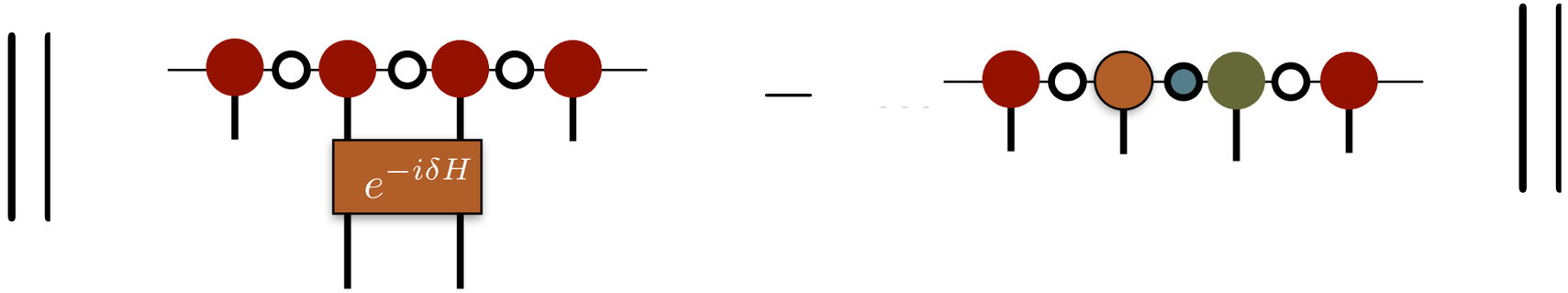
G. Vidal, Phys. Rev. Lett. 91, 147902 (2003)

なぜ TEBD は高精度なのか？

- 高精度計算のためには正準形が重要
  - λがSchmidt係数であれば, 系の残りの部分の情報をすべて含んでいる



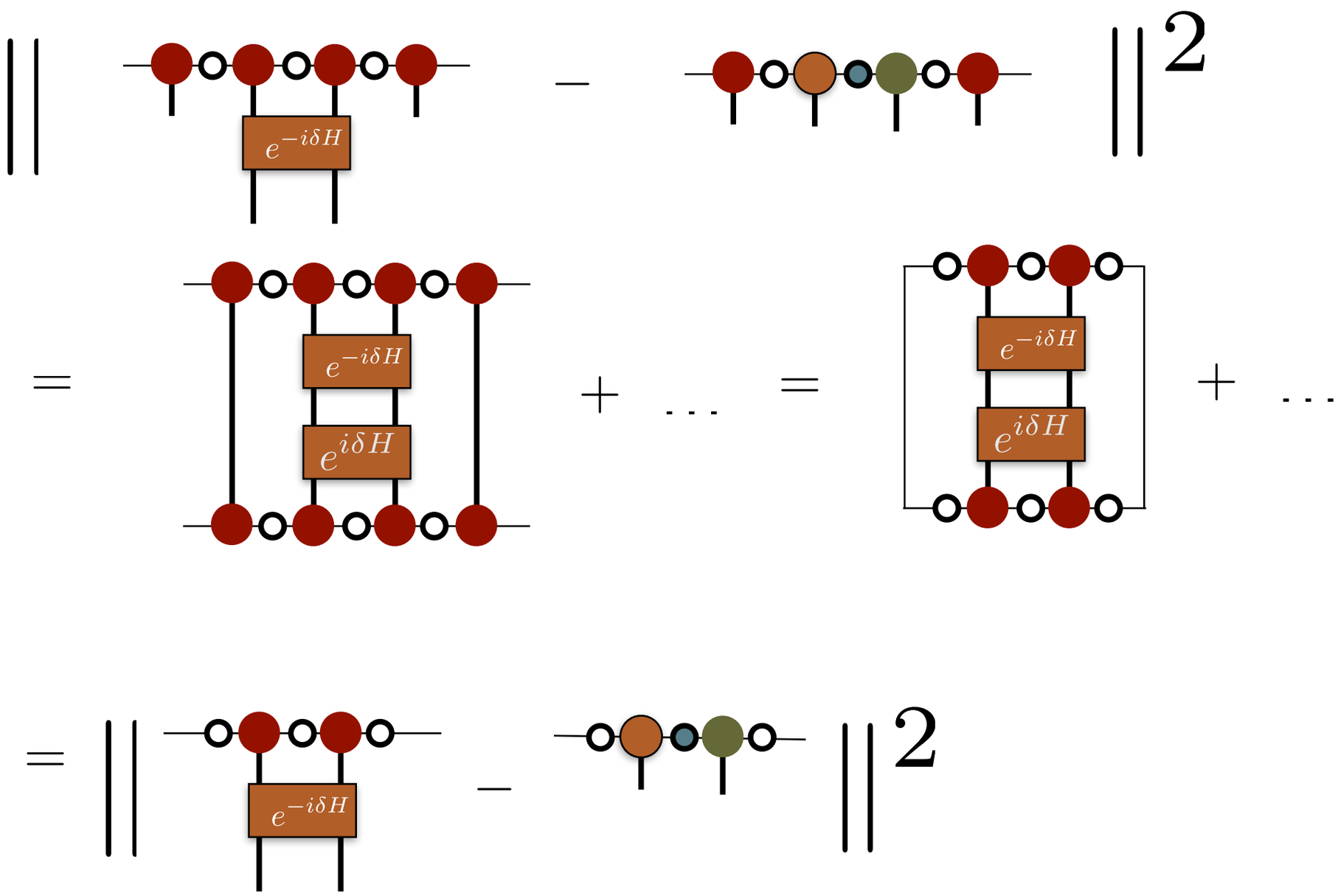
- カノニカル形式のおかげで, TEBD アルゴリズムが 2 つの量子状態の距離を最小化することを示せる



- 局所SVDに基づく切り捨ては、MPSの一部だけを見ていても大域的に最適になりうる

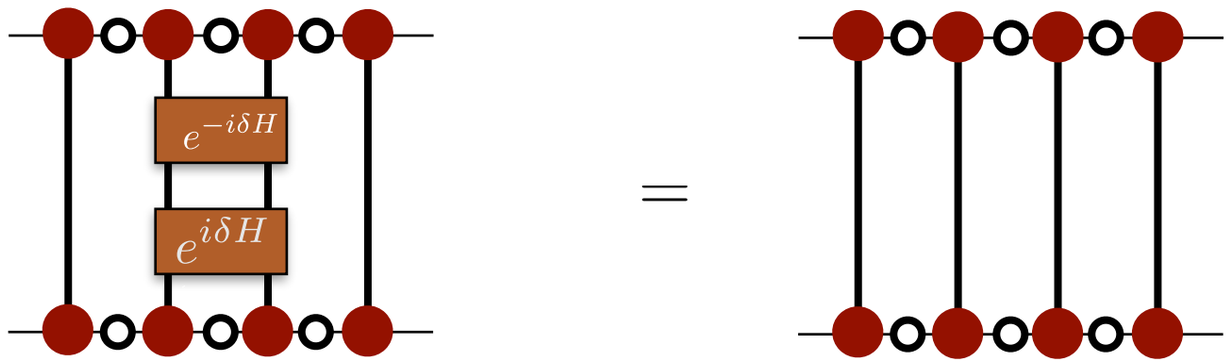
MPS 同士の距離

- 局所SVDによる低ランク近似が大域的な距離最小化に対応する



なぜ TEBD は高精度なのか？

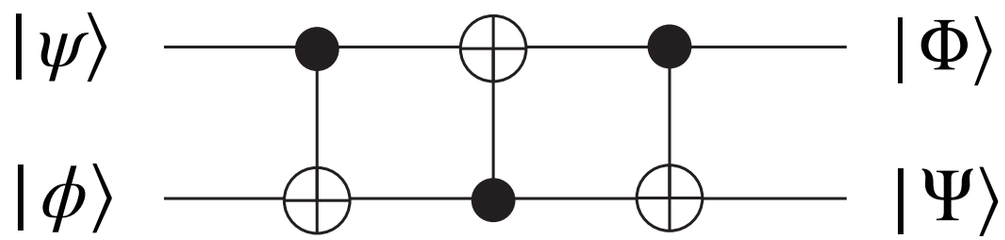
- 演算子がユニタリなら, 近似の後でも MPS はカノニカル形式を保つ



- ユニタリ演算子は、他の Schmidt 係数に影響しない
- 初期MPSを正準形にしておけば、TEBD アルゴリズムはそれをほぼ保つ
- TEBDは「ほぼ」大域的最適である

長距離相互作用

- 離れた量子ビット間の演算をどう適用するか？
  - SWAPゲートを使って, 最近接相互作用へ変換する



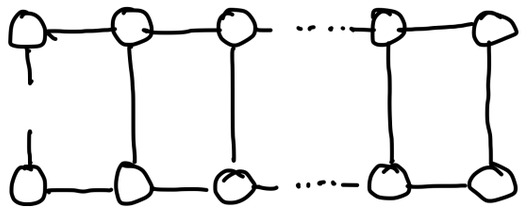
MPS状態からのサンプリング

MPSからの状態のサンプリング

- 2^n次元空間でのサンプリングは難しい
  - $P(s_1, s_2, \cdots, s_n) = |\langle s_1 s_2 \cdots s_n | \Psi \rangle|^2 / \langle \Psi | \Psi \rangle$
  - タワーサンプリング: メモリコスト $\sim O(2^n)$, 計算量 $\sim O(2^n)$

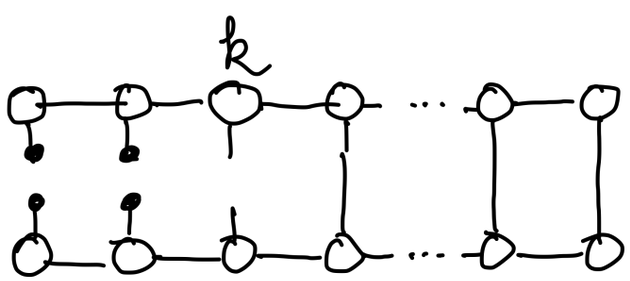
- いったん(近似)MPS 表現が得られれば周辺分布を使って状態を逐次にサンプルできる
  - 周辺分布:

$$P(s_1) = \sum_{s_2, \cdots, s_n} P(s_1, s_2, \cdots, s_n)$$



  - 条件付き分布:

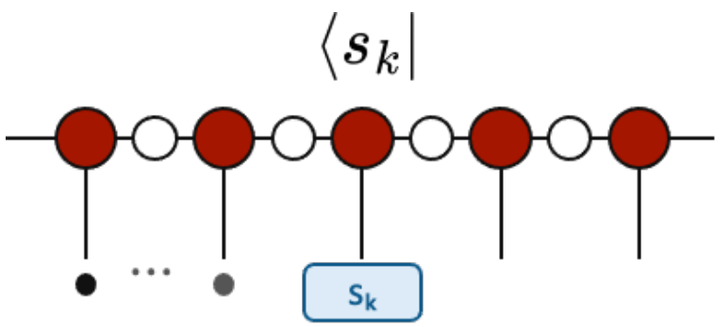
$$P(s_k | s_1, \cdots, s_{k-1}) = \frac{\sum_{s_{k+1}, \cdots, s_n} P(s_1, s_2, \cdots, s_n)}{\sum_{s_k, \cdots, s_n} P(s_1, s_2, \cdots, s_n)}$$



測定後の状態

- 結果 s_k が得られたら、状態を射影して正規化する

$$|\Psi\rangle \longrightarrow \frac{P_{s_k}^{(k)}|\Psi\rangle}{\sqrt{\langle\Psi|P_{s_k}^{(k)}|\Psi\rangle}}$$

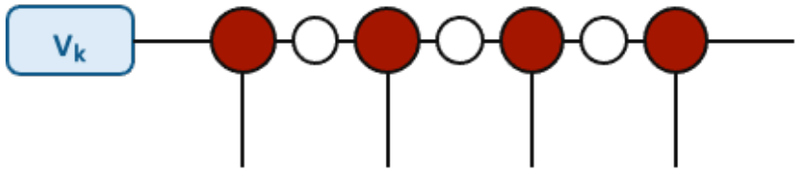


- 左カノニカル形式では、条件付き確率と更新が局所的に書ける
  - 測定後の残りの状態も再びMPSになる

$$p_k(s) = P(s_k = s \mid s_1, \ldots, s_{k-1}) = \|v_{k-1}A_k[s]\|^2, \qquad v_0 = 1$$

残りの状態は再び MPS

$$v_k = \frac{v_{k-1}A_k[s_k]}{\sqrt{p_k(s_k)}}$$



- これを k=1, ..., N まで繰り返すと、1本のビット列が得られる

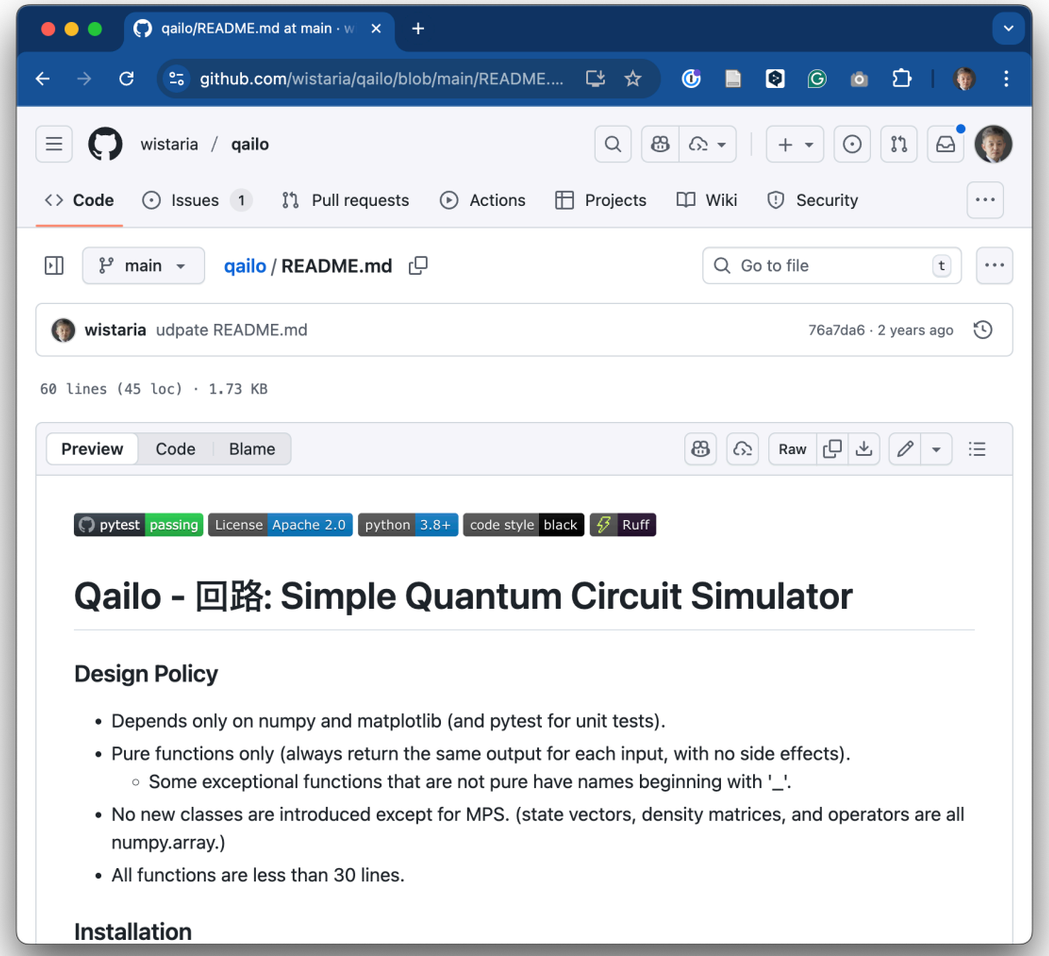
逐次サンプル→更新→次へ

Qailo

Qailo - 回路: Simple Quantum Circuit Simulator

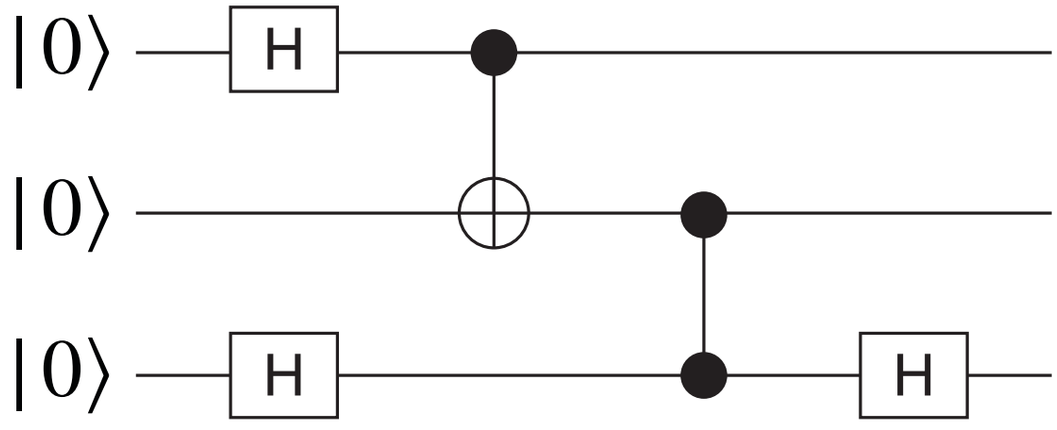
- オープンソースソフトウェア on GitHub
  - https://github.com/wistaria/qailo
  - 状態ベクトル(Schrödinger)シミュレーションとMPS (TEBD)シミュレーション(混合カノニカル形式)をサポート (Vidalカノニカル形式もサポート予定)

- Qailoの設計方針
  - numpyとmatplotlibのみに依存(ユニットテストにはpytestも使用)
  - 純粋関数のみ(各入力に対して常に同じ出力を返し、副作用なし)
  - 純粋でない例外的な関数は名前を“_”で始める
  - MPS を除き新規クラスは導入しない(状態ベクトル、密度行列、演算子は全てnumpy.array)
  - 全関数を30行未満とする



Qailo: 最初の例題

- 3量子ビットの量子回路



  - https://github.com/wistaria/qailo/blob/main/example/simple.py
- 状態ベクトルと演算子の実装
  - n量子ビット状態ベクトル: (n+1)本足テンソル (numpy ndarray)
    - https://github.com/wistaria/qailo/blob/main/src/qailo/state_vector/state_vector.py
  - m量子ビット演算子: (2m)本足テンソル (numpy ndarray)
    - https://github.com/wistaria/qailo/blob/main/src/qailo/operator/one_qubit.py

Qailo: 状態ベクトルシミュレーション

- 状態ベクトルへの演算
  - https://github.com/wistaria/qailo/blob/main/src/qailo/state_vector/apply.py
  - np.einsum(A, [0, 2], B, [2, 1], [0, 1]) 形式を使用
  - np.einsum("ij,jk->ik", A, B) は使わない
- それ以外の機能
  - 一次元ベクトルへの変換
  - 密度行列の生成
  - フィデリティ・確率の計算
  - (部分)トレース
  - など
- サンプルコード
  - QFT(量子フーリエ変換)とiQFT(逆変換)
  - QPE(量子位相推定)
  - など

Hands-on

- Hands-on 1
  - MPSで表現されているN量子ビットGHZ状態のノルムを2通りの方法で計算する
    - 1. MPSを縮約して全系の状態を表すN本足のテンソルを作り、そのノルムを計算する
    - 2. ブラを表すMPSとケットを表すMPSを縮約するテンソルネットワーク(はしご状)を考え、端から順に縮約する
    - それぞれの実行時間のN依存性を計測する

- Hands-on 2
  - 32_tebd を修正して、Vidalカノニカル形式を用いたTEBDを実装する
  - カノニカル形式を使わない場合(32_tebdのmps1)と比較して、どれくらいフィデリティが改善するか確認する